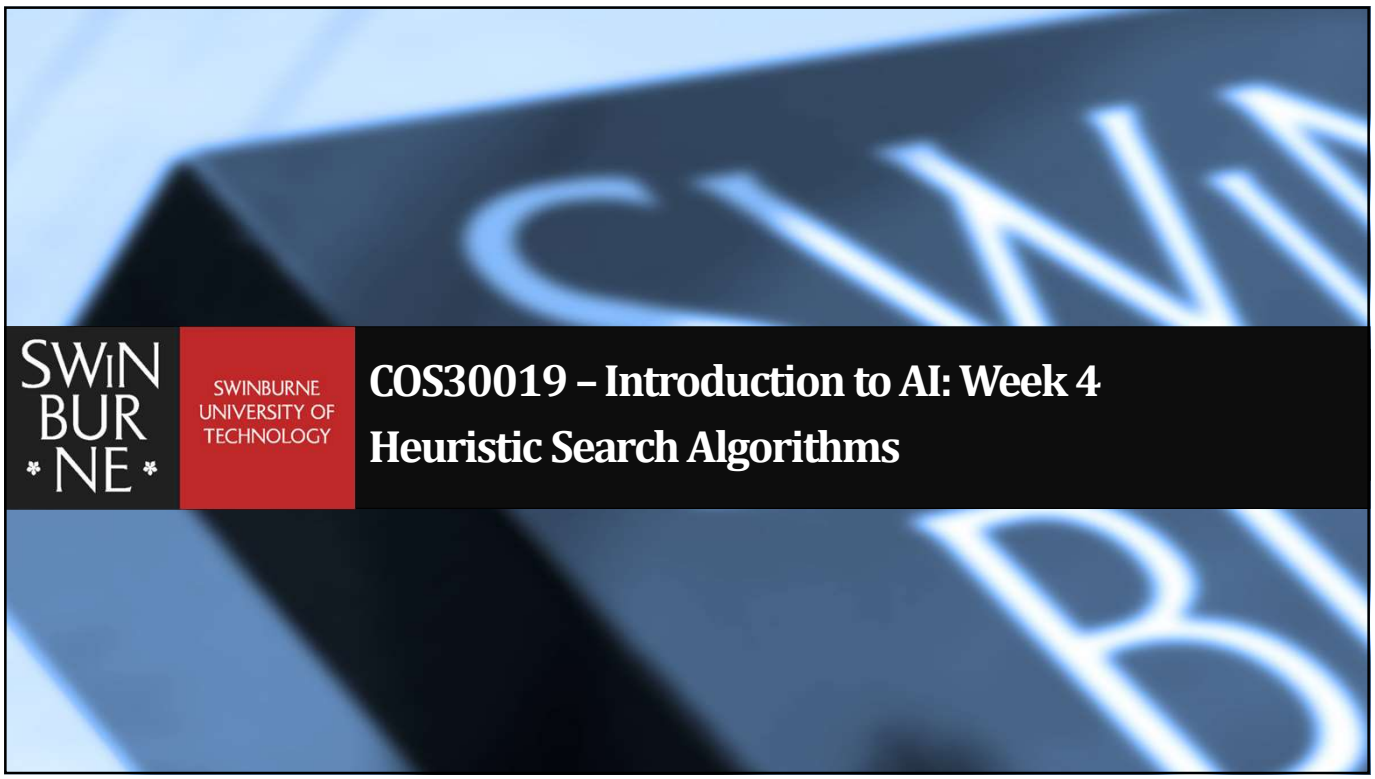


1

Previously: tree-search

function TREE-SEARCH(*problem, frontier*) **return** a solution or failure

frontier ← INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *frontier*)

[*explored* ← {}]

loop do

if EMPTY?(*frontier*) **then return** failure

node ← REMOVE-FIRST(*frontier*)

if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds

then return SOLUTION(*node*)

frontier ← INSERT-ALL(EXPAND(*node*, *problem*), *frontier,* [*explored*])

A strategy is defined by picking *the order of node expansion*

2

Search strategies

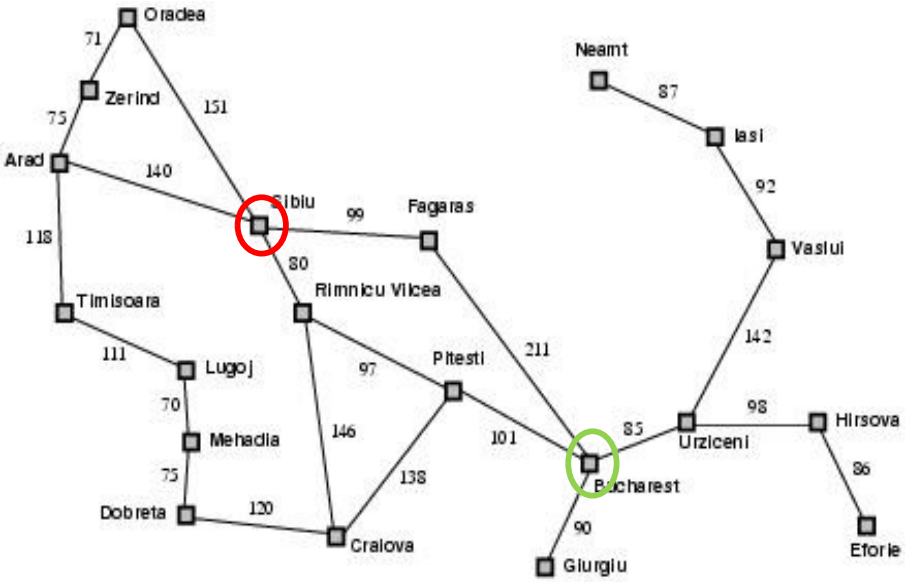
- Blind search – BFS, DFS, uniform cost
  - no notion of the "right direction"
  - can only recognize goal once it's achieved

Example: Route finding

Initial state = Sibiu
Goal= Bucharest

From Sibiu, **DFS** & **BFS** would not treat **Arad** or **Oradea** any different from **Fagaras** or **Rimnicu Vilcea**.

However, most people would not consider **Arad** or **Oradea** when figuring out a route from Sibiu to Bucharest



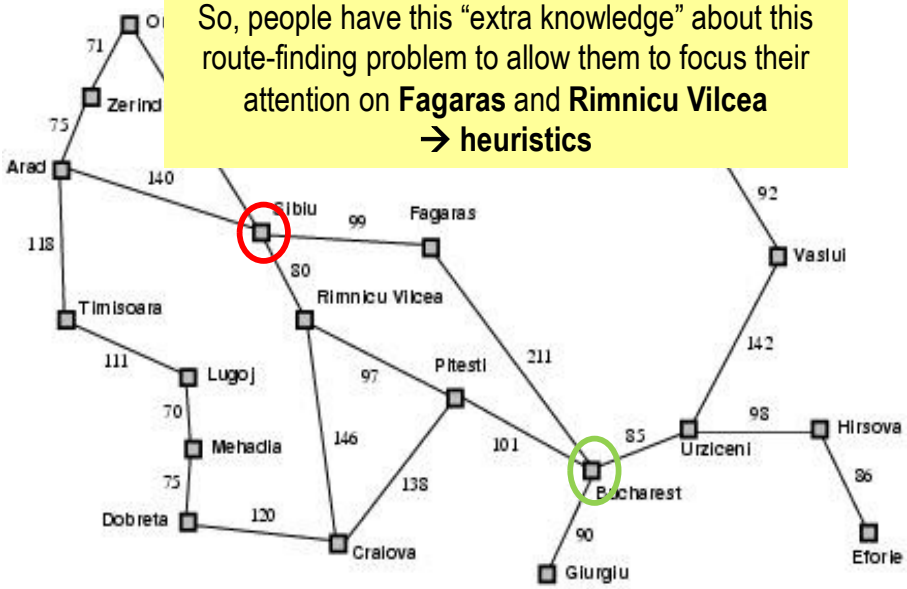
Example: Route finding

Initial state = Sibiu
Goal= Bucharest

From Sibiu, **DFS** & **BFS** would not treat **Arad** or **Oradea** any different from **Fagaras** or **Rimnicu Vilcea**.

However, most people would not consider **Arad** or **Oradea** when figuring out a route from Sibiu to Bucharest

So, people have this "extra knowledge" about this route-finding problem to allow them to focus their attention on **Fagaras** and **Rimnicu Vilcea**
→ **heuristics**



SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Search strategies

- Blind search – BFS, DFS, uniform cost
  - no notion of the "right direction"
  - can only recognize goal once it's achieved
- Heuristic search – we have rough idea of how good various states ar

Def: *A rational agent chooses whichever action that maximizes the expected value of the performance measure given the percept sequence … owledge.*
(From Week 1)

Prior environment knowledge
→ heuristics

In this lecture…

- An informed strategy uses problem-specific knowledge to pick the "more promising" node
- Which search strategies?
  - Best-first search and its variants
- Heuristic functions
- Local search and optimization
  - ☑Hill climbing,
  - local beam search,
  - genetic algorithms,
  - …

A heuristic function

- [dictionary]*"A rule of thumb, simplification, or educated guess that reduces or limits the search for solutions in domains that are difficult and poorly understood."*
  - ☐$h(n)$ = estimated cost to get from node n to a goal node.

    i.e., $h(n)$ tells us how close (approximately) node n is to goal
  - ☐If n is goal then $h(n)=0$

SWIN BURNE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

A heuristic function

- [dictionary]*"A rule of thumb, simplification, or educated guess that reduces or limits the search for solutions in domains that are difficult and poorly understood."*
  - $h(n)$ = estimated cost to get from node n to a goal node.
  - If n is go

First idea: node with the **lowest estimated cost** is the **most promising one**
That is, the *frontier* can be implemented as a priority queue of nodes such that nodes are **ordered** according to $h(n)$ from **low to high**.

A heuristic function

- [dictionary]*"A rule of thumb, simplification, or educated guess that reduces or limits the search for solutions in domains that are difficult and poorly understood."*
  - $h(n)$ = esti
  - If n is go

But, is $h(n)$ the only way to determine how promising a node n is??

Hint: There are more than one way but they usually use $h(n)$ (because at the end of the day, $h(n)$ encodes our "extra knowledge" about this problem)

Best-first search

- General approach of informed search:
  - Best-first search: node is selected for expansion based on an *evaluation function* $f(n)$
- Idea: evaluation function measures the "overall" cost to goal via node *n*.
  - Choose node which *looks most promising*

Best-first search

- General approach of informed search:
  - Best-first search: node is selected for expansion based on an *evaluation function* $f(n)$
- Idea: evaluation function measures the "overall" cost to goal via node *n*.
  - Choose node which *looks most promising*

That is, each way to determine how promising a node n is would give you an evaluation function $f(n)$

Best-first search

- General approach of informed search:
  - Best-first search: node is selected for expansion based on an *evaluation function* $f(n)$
- Idea: evaluation function measures the "overall" cost to goal via node *n*.
  - Choose node which *looks most promising*
- Implementation:
  - *frontier* is a priority queue sorted in increasing order of the evaluation function.
  - Special cases: **greedy search**, **A* search**

Best-first search

- General approach of informed search:
  - Best-fir
  $f(n)$
- Idea: eval
  - Choose
- Implemen
  - *frontie*
  - Special

[SPOILER ALERT]

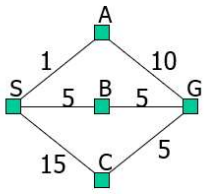
Some of the most used *evaluation functions*:

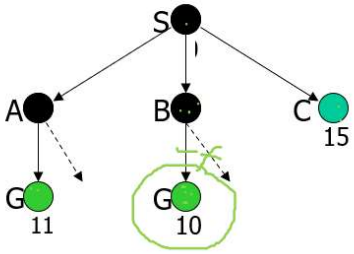
- f(n) = g(n)
  - ❑ Uniform-cost search (UCS)
- f(n) = h(n)
  - ❑ Greedy Best First Search (GBFS)
- f(n) = g(n) + h(n)
  - ❑ A* Search (AS)

Flash back - UCS

Uniform-Cost Search (UCS) Strategy

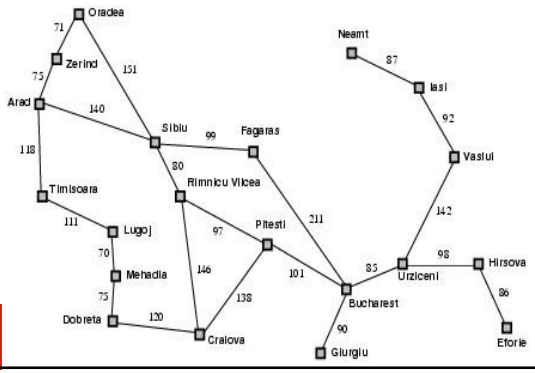
- Each step has some cost $\geq \varepsilon > 0$.
- The cost of the path (from the root node) to each frontier node N is
  $g(N) = \Sigma$ costs of all steps from root to N.
- The goal is to generate a solution path of minimal cost.
- The queue FRONTIER is sorted in increasing cost.





Example: Route-finding in Romania

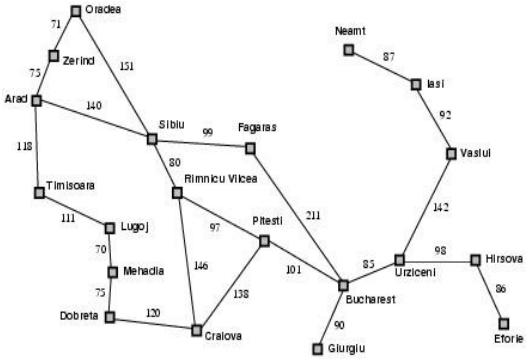
- h_{SLD}=straight-line distance (between a state and goal) heuristic.
- h_{SLD} can **NOT** be computed from the problem description itself



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Dobreta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Route-finding in Romania - GBFS

- h_{SLD}=straight-line distance (between a state and goal) heuristic.
- h_{SLD} can **NOT** be computed from the problem description itself



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Dobreta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

If we define: $f(n)=h_{SLD}(n)$

i.e., Expand node that is closest to goal
= **Greedy best-first search**

SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

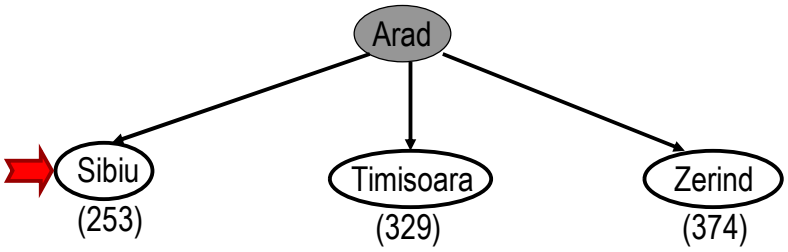
Greedy best-first search example

(366)

- Assume that we want to use greedy search to solve the problem of traveling from Arad to Bucharest.
- The initial state=Arad

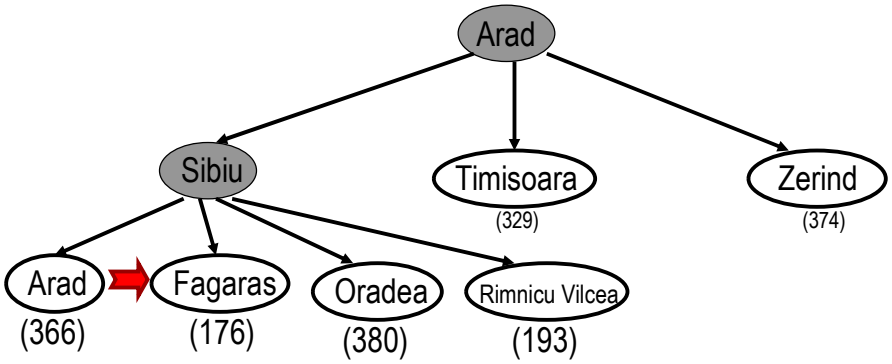
SWIN BUR NE — CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Greedy best-first search example



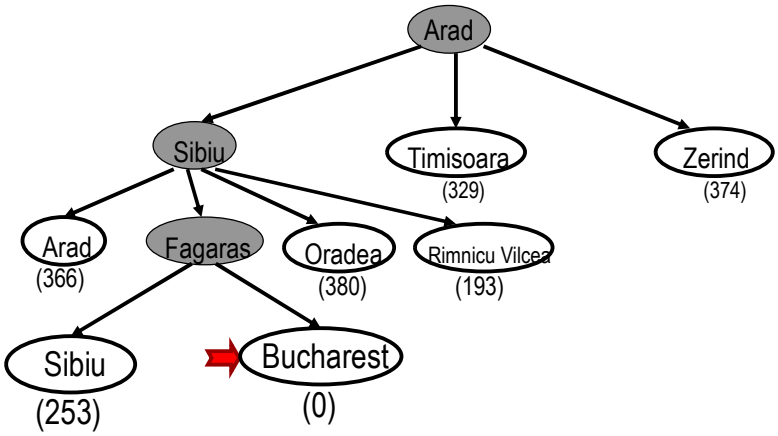
- The first expansion step produces:
  - Sibiu, Timisoara and Zerind
- Greedy best-first will select Sibiu.

Greedy best-first search example



- If Sibiu is expanded we get:
  - Arad, Fagaras, Oradea and Rimnicu Vilcea
- Greedy best-first search will select: Fagaras

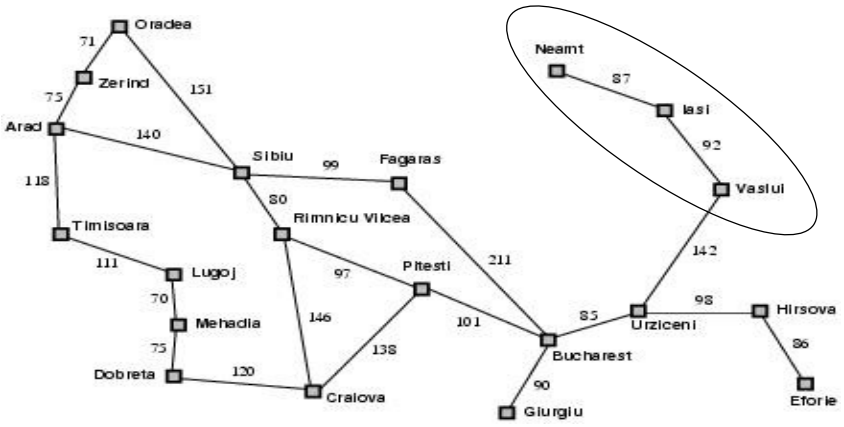
Greedy best-first search example



- If Fagaras is expanded we get:
  - Sibiu and Bucharest
- Goal reached !!
  - Yet not optimal (see Arad, Sibiu, Rimnicu Vilcea, Pitesti)

Greedy best-first search, evaluation

- Completeness: NO (cfr. DF-search)
  - Check on repeated states
  - Minimizing ***h(n)*** can result in false starts, e.g. Iasi to Fagaras.



Greedy best-first search, evaluation

- Completeness: NO (cfr. DF-search)
- Time complexity? $O(b^m)$
  - Cfr. Worst-case DF-search

  (with m is maximum depth of search space)
  - Good heuristic can give dramatic improvement.

Greedy best-first search, evaluation

- Completeness: NO (cfr. DF-search)
- Time complexity: $O(b^m)$
- Space complexity: $O(b^m)$
  - Keeps all nodes in memory

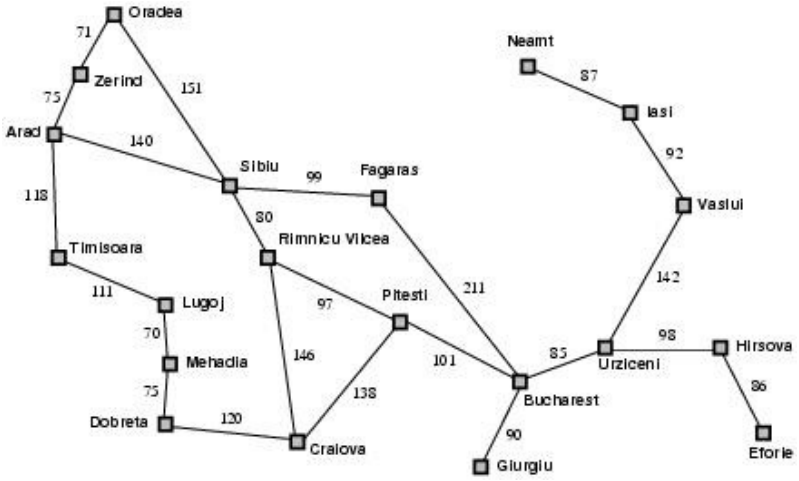
Greedy best-first search, evaluation

- Completeness: NO (cfr. DF-search)
- Time complexity: $O(b^m)$
- Space complexity: $O(b^m)$
- Optimality? NO
  - Same as DF-search

A* search

- Best-known form of best-first search.
- Important AI algorithm developed by Fikes and Nilsson in early 70s. Originally used in Shakey robot.
- Idea: avoid expanding paths that are already expensive.
- Evaluation function $f(n)=g(n) + h(n)$
  - $g(n)$ the cost (so far) to reach the node.
  - $h(n)$ estimated cost to get from the node to the goal.
  - $f(n)$ estimated total cost of path through n to goal.

Romania example

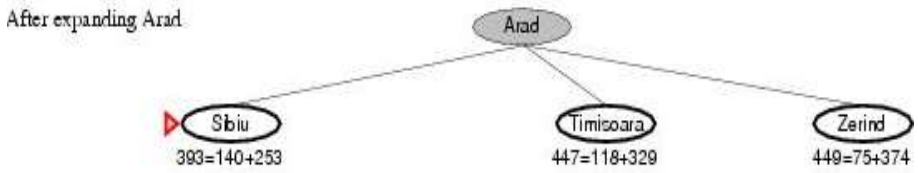


A* search example

(a) The initial state

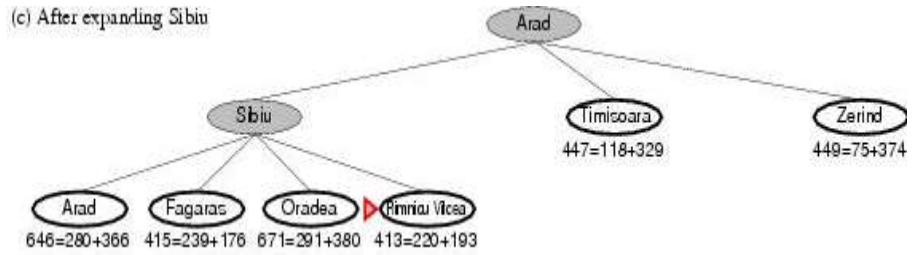
- Find Bucharest starting at Arad
  - f(Arad) = g(Arad)+h(Arad)=0+366=366

A* search example



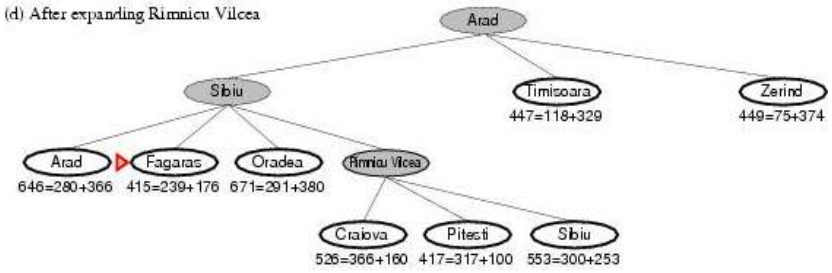
- Expand Arrad and determine *f(n)* for each node
  - f(Sibiu)=g(Sibiu)+h(Sibiu)=140+253=393
  - f(Timisoara)=g(Timisoara)+h(Timisoara)=118+329=447
  - f(Zerind)=g(Zerind)+h(Zerind)=75+374=449
- Best choice is Sibiu

A* search example



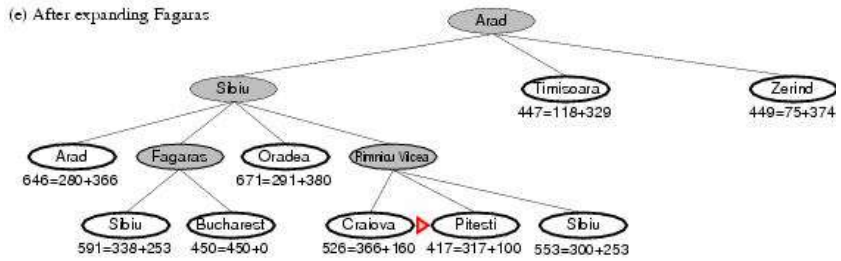
- Expand Sibiu and determine *f(n)* for each node
  - f(Arad)=g(Arad)+h(Arad)=280+366=646
  - f(Fagaras)=g(Fagaras)+h(Fagaras)=239+179=415
  - f(Oradea)=g(Oradea)+h(Oradea)=291+380=671
  - f(Rimnicu Vilcea)=g(Rimnicu Vilcea)+h(Rimnicu Vilcea)=220+192=413
- Best choice is Rimnicu Vilcea

A* search example



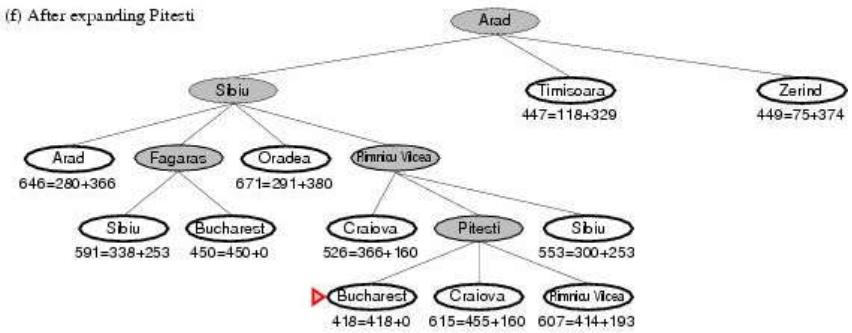
- Expand Rimnicu Vilcea and determine *f(n)* for each node
  - f(Craiova)=g(Craiova)+h(Craiova)=360+160=526
  - f(Pitesti)=g(Pitesti)+h(Pitesti)=317+100=417
  - f(Sibiu)=g(Sibiu)+h(Sibiu)=300+253=553
- Best choice is Fagaras

A* search example



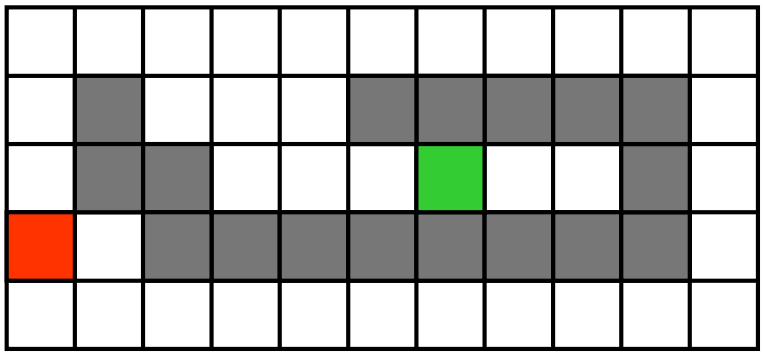
- Expand Fagaras and determine *f(n)* for each node
  - f(Sibiu)=g(Sibiu)+h(Sibiu)=338+253=591
  - *f(Bucharest)=g(Bucharest)+h(Bucharest)=0+450=450*
- Best choice is Pitesti !!!

A* search example



- Expand Pitesti and determine *f(n)* for each node
  - *f(Bucharest)=g(Bucharest)+h(Bucharest)=0+418=418*
- Best choice is Bucharest !!!
  - Optimal solution (under some conditions on *h(n)* – to be discussed later)
- Note values along optimal path !!

Another example: Robot Navigation



Robot Navigation: Greedy best-first search

f(N) = h(N), with h(N) = Manhattan distance to the goal

8	7	6	5	4	3	2	3	4	5	6
7		5	4	3						5
6			3	2	1	0	1	2		4
7	6									5
8	7	6	5	4	3	2	3	4	5	6

Robot Navigation : Greedy best-first search

f(N) = h(N), with h(N) = Manhattan distance to the goal

8	7	6	5	4	3	2	3	4	5	6
7		5	4	3						5
6			3	2	1	0	1	2		4
7	6									
8	7	6	5	4	3	2	3	4	5	6

What happened???

Robot Navigation : A* search

f(N) = g(N)+h(N), with h(N) = Manhattan distance to goal

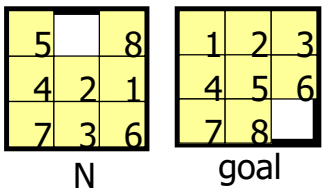
8+3	7+4	6+3	5+6	4+7	3+8	2+9	3+10	4	5	6
7+2		5+6	4+7	3+8						5
6+1			3	2+9	1+10	0+11	1	2		4
7+0	6+1									5
8+1	7+2	6+3	5+4	4+5	3+6	2+7	3+8	4	5	6

A* search, evaluation

- Completeness: YES (Unless there are infinitely many nodes n with $f(n) \leq f(G)$)
- Time complexity: (exponential with path length)
- Space complexity: (all nodes are stored)
- **Optimality**: YES; under two conditions:
  - The heuristic function $h(n)$ is admissible (never overestimates the true cost)
  - $h(n)$ is consistent (for every node n and successor n' with step cost c, $h(n) \leq h(n') + c$)
  - *Formal analyses can be found in AIMA*

Heuristic Function

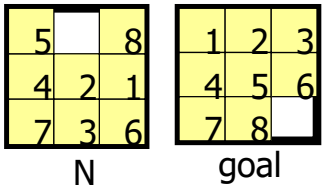
- Function h(*n*) that estimates the cost of the cheapest path from node *n* to goal node.
- Example: 8-puzzle



h(N) = number of misplaced tiles
= 6

Heuristic Function

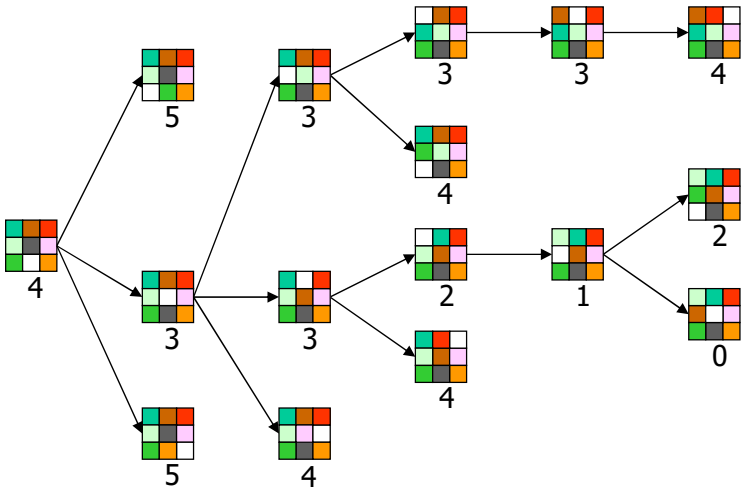
- Function h(N) that estimate the cost of the cheapest path from node N to goal node.
- Example: 8-puzzle



h(N) = sum of the distances of every tile to its goal position
= 2 + 3 + 0 + 1 + 3 + 0 + 3 + 1
= 13

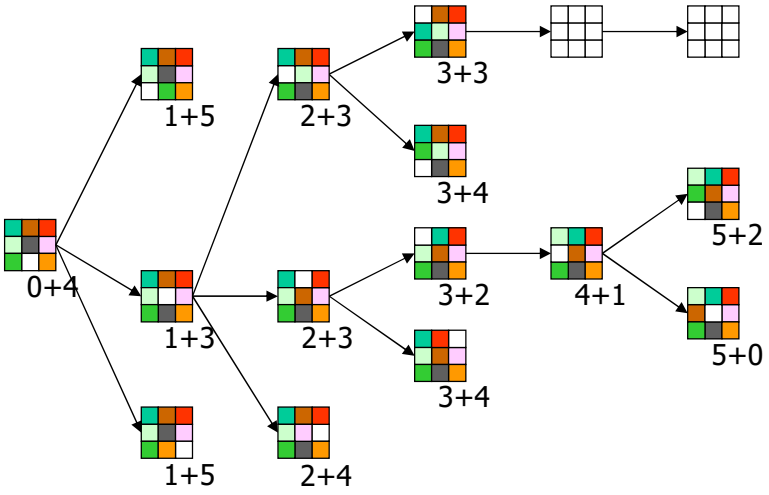
8-Puzzle

f(N) = h(N) = number of misplaced tiles



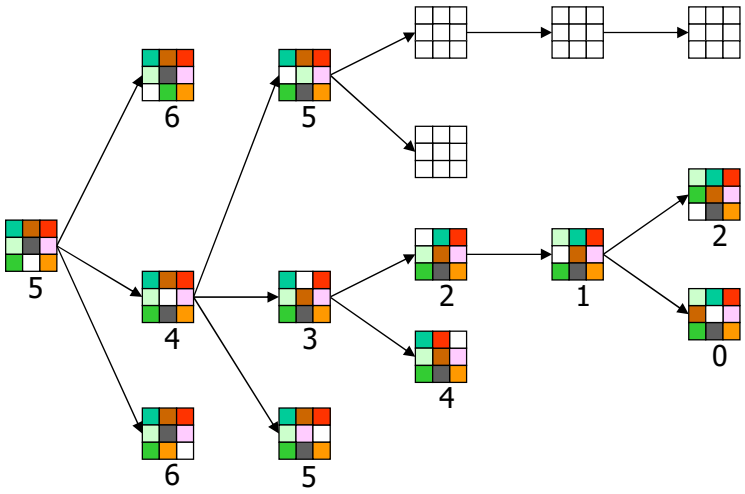
8-Puzzle

f(N) = g(N) + h(N)
with h(N) = number of misplaced tiles



8-Puzzle

f(N) = h(N) = Σ distances of tiles to goal



8-Puzzle

5		8
4	2	1
7	3	6

N

1	2	3
4	5	6
7	8	

goal

- $h_1(N)$ = number of misplaced tiles = 6 is admissible
- $h_2(N)$ = sum of distances of each tile to goal = 13
  is admissible
- $h_3(N)$ = (sum of distances of each tile to goal)
  + 3 x (sum of score functions for each tile) = 49
  is not admissible

About Heuristics

- Heuristics are intended to orient the search along promising paths
- The time spent computing heuristics must be recovered by a better search
- After all, a heuristic function could consist of solving the problem; then it would perfectly guide the search
- Deciding which node to expand is sometimes called meta-reasoning
- Heuristics may not always look like numbers and may involve large amount of knowledge

SWIN BUR NE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

What's the Issue?

- Search is an iterative local procedure
- Good heuristics should provide some global look-ahead (at low computational cost)

SWIN BUR NE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Another approach…

- for optimization problems
  - rather than constructing an optimal solution from scratch, start with a suboptimal solution and iteratively improve it
- Local Search Algorithms
  - Hill-climbing or Gradient descent
  - Potential Fields
  - Simulated Annealing
  - Genetic Algorithms, others…

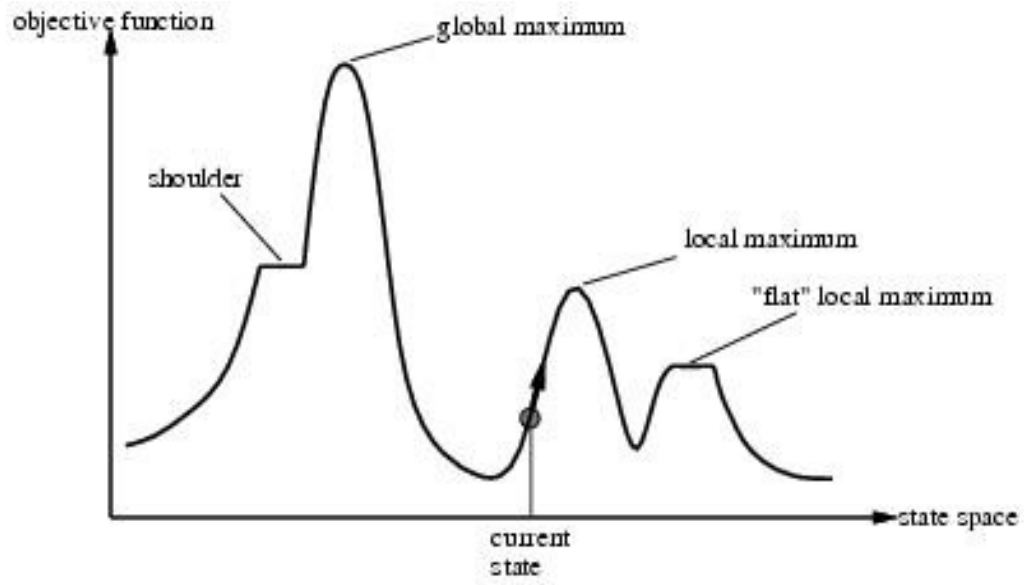
SWIN BUR NE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

Hill-climbing search

- If there exists a successor s for the current state n such that
  - h(s) < h(n)
  - h(s) <= h(t) for all the successors t of n,
- then move from n to s. Otherwise, halt at n.
- Looks one step ahead to determine if any successor is better than the current state; if there is, move to the best successor.
- Similar to Greedy search in that it uses h, but does not allow backtracking or jumping to an alternative path since it doesn't "remember" where it has been.
- Not complete since the search will terminate at "local maxima," "plateaus," and "ridges."

SWIN BUR NE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

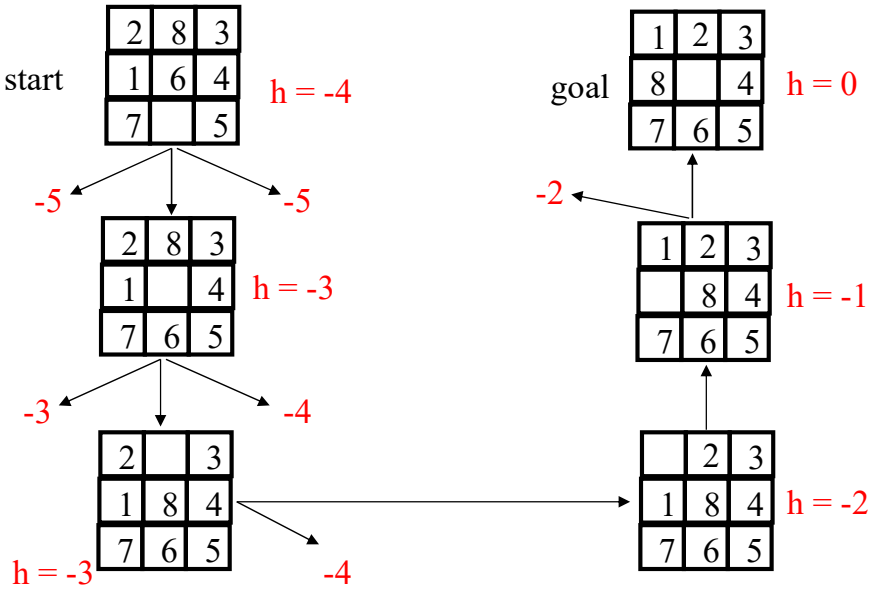
Local search and optimization



Drawbacks of hill climbing

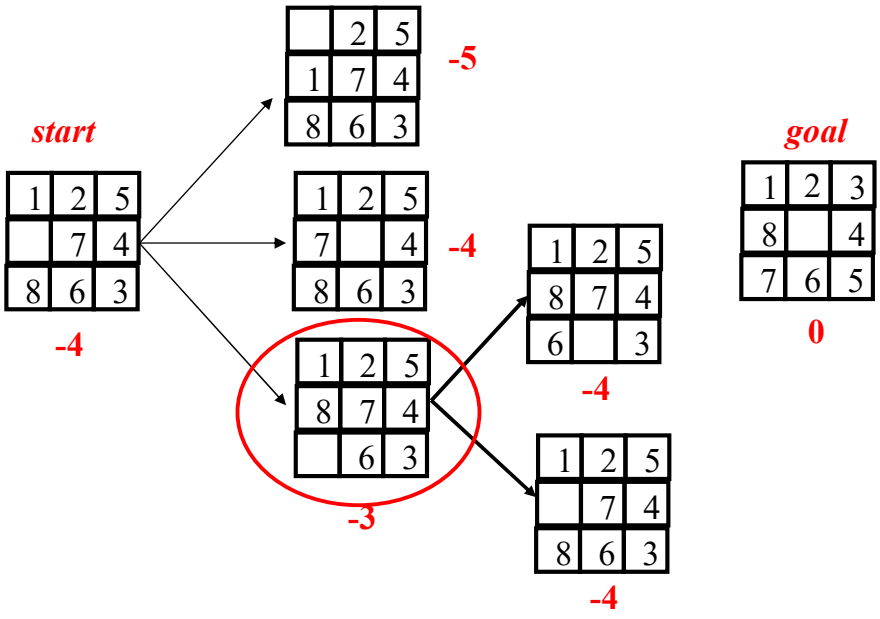
- Problems:
  - **Local Maxima**: peaks that aren't the highest point in the space
  - **Plateaus:** the space has a broad flat region that gives the search algorithm no direction (random walk)
- Remedy:
  - Introduce randomness
    - Random restart.
- Some problem spaces are great for hill climbing and others are terrible.

Hill climbing example



f(n) = -(number of tiles out of place)

Example of a local maximum



When to Use Search Techniques?

- The search space is small, and
  - There is no other available techniques, or
  - It is not worth the effort to develop a more efficient technique
- The search space is large, and
  - There is no other available techniques, and
  - There exist "good" heuristics

Summary

- Heuristic function
- Best-first search
  - Greedy best-first search
  - A* (with admissible and consistent heuristic)
- Heuristic accuracy
- Optimization problems
  - Hill climbing